

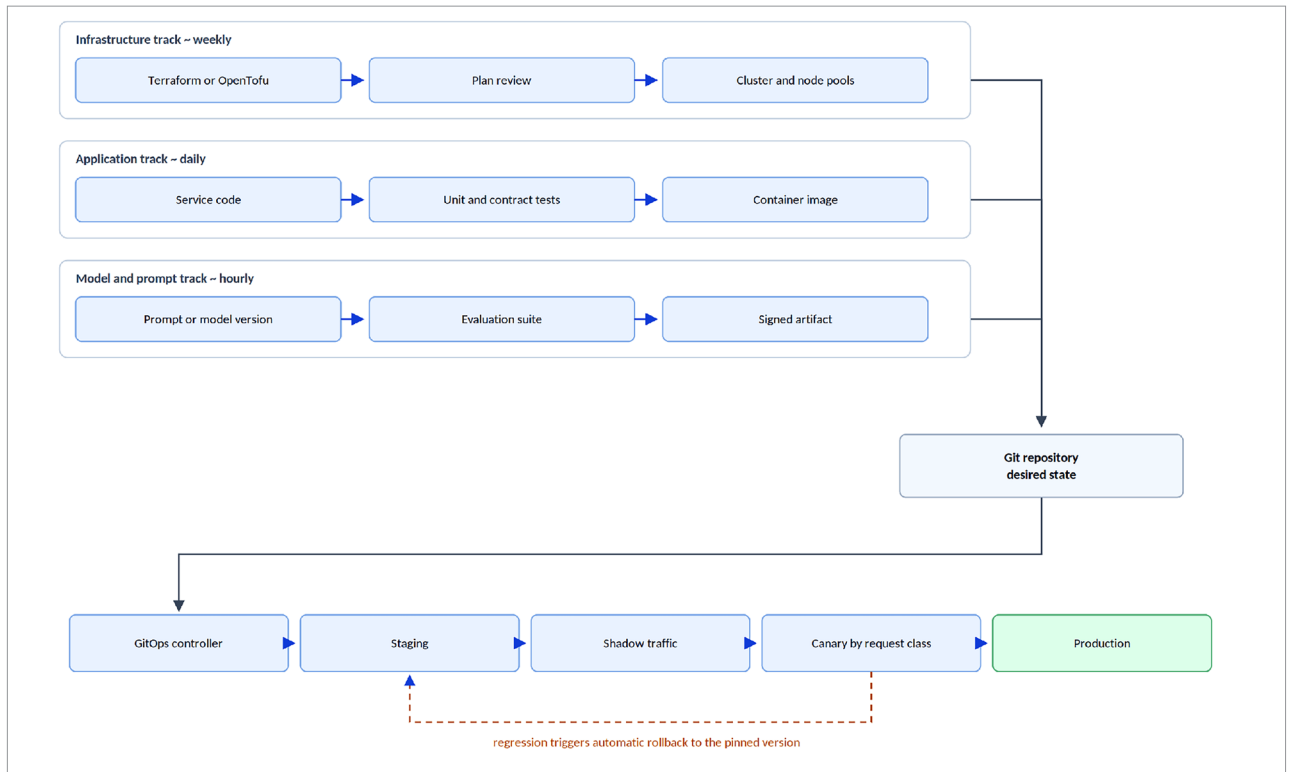


Figure 6: Three tracks, one desired state. The model and prompt track moves fastest and therefore needs the strictest automated gates.

Level 4 matters more than it seems to. If the product has no non-AI path, an inference outage is a full product outage, which is a decision worth making deliberately.

Regulated deployments need one more thing: an immutable record linking the request, prompt and model version, retrieved document IDs, tool calls, output, and any approver.

Which failure patterns show up most often?

Most of these are observability failures first. Each was detectable earlier with a signal the platform was not collecting.

Pattern	Underlying cause	Fix
Green dashboards, unhappy users	Streaming stalls hidden by end-to-end averages	Measure TTFT and inter-token latency separately
High utilisation, low output	Small batches, memory bandwidth bound	Continuous batching, tune KV cache size
Retrieval quality collapse	Embedding model changed without reindex	Version the embedding model with the index
Provider outage takes everything down	Fallback configured but never tested	Scheduled failover exercises

Where should you start?

Sequence matters more than completeness:

- Put the gateway in first, because it gives you identity, metering, routing, and a kill switch in one component, and every later control has somewhere to live.
- Instrument to the genAI conventions next, with content capture off.
- Build the evaluation suite third, since it is the prerequisite for routing, caching, and any model change you want to make safely.
- Optimise cost last, because caching and routing without evaluations is optimisation without a safety net.

The honest caveat is that this order is slower than shipping against a provider API and dealing with the platform later. Plenty of products started that way. The difference shows up at the third or fourth AI service, when nobody can say which team is spending what, or what an agent could reach if asked nicely enough.

The discipline that made microservices manageable still applies. It just has to account for workloads that cost per token, hold state in GPU memory, treat their input as instruction, and answer differently to the same question twice. **END** 🐧

By: Raj Patel

The author is a SaaS enthusiast and technical content creator.